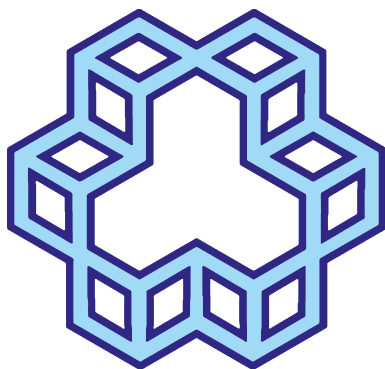




دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی کامپیوتر-

پایان نامه دوره کارشناسی

مهندسی کامپیوتر

عنوان

طراحی و پیاده سازی یک سرور DNS

نگارش

هدیه طهمورثی و سپهر قارداشی

استاد راهنما

دکتر کعبه یعقوبی

زمستان ۱۴۰۴

فهرست مطالب

۲	۱ مقدمه
	section ۱.۱ مقدمه ۲
۳	۲.۱ تعریف مسئله
	section ۳.۱ اهداف تحقیق ۳
۴	۱.۳.۱ اهداف عملکردی
	subsection ۳.۱ اهداف غیر عملکردی ۴
۵	۴.۱ ساختار گزارش
	chapter ۲ مبانی نظری و پیشینه تحقیق ۶
۶	۱.۲ مقدمه
	section ۲.۲ سامانه نام دامنه ۶ (DNS)
۷	۱.۲.۲ سلسله مراتب نام ها
	subsection ۲.۲ ساختار بسته DNSv
۸	۳.۲ ملاحظات پروتکل انتقال
	subsection ۳.۲ پروتکل داده نگار کاربر ۸ (UDP)
۸	۲.۳.۲ محدودیت اندازه بسته
	section ۴.۲ تهدیدات امنیتی ۸
۹	۱.۴.۲ حملات انکار سرویس (DoS)
	subsection ۴.۲ حملات تقویتی ۹
۹	۵.۲ استراتژی های ذخیره سازی نهان (Caching)
	subsection ۵.۲ زمان حیات ۹
۱۰	۲.۵.۲ سیاست های تخلیه
	section ۶.۲ مبانی الگوریتمی ۱۰
۱۰	۱.۶.۲ جداول درهم سازی
	subsection ۶.۲ الگوریتم پنجره لغزان ۱۱
۱۲	۳ طراحی و پیاده سازی
	section ۱.۳ مقدمه ۱۲
۱۲	۲.۳ معماری سیستم
	subsection ۲.۳ ساختار پروژه ۱۳
۱۴	۳.۳ لایه شبکه: سرور چندنخی
	subsection ۳.۳ واکار: مدل نخ-به-ازای-درخواست ۱۴
۱۵	۲.۳.۳ تحمل خرابی (Fault Tolerance)
	section ۴.۳ هسته پردازشی: خط لوله حل نام ۱۵
۱۶	۱.۴.۳ ارسال ایزوله (Socket Isolation)
	section ۵.۳ مدیریت داده ها: ناحیه محلی و کش ۱۶
۱۶	۱.۵.۳ پشتیبانی از رکوردهای متنوع

	۵.۳ subsection	گلش هوشمند و ایمن ۱۷
۱۸	۶.۳	امنیت و کنترل ترافیک: پیاده سازی دقیق
	۶.۳ subsection	محدود سازی نرخ (Rate Limiting) ۱۸
۲۰	۲۰.۶.۳	سیاست گذاری و تطبیق پسوندی
	۷.۳ section	سیستم ثبت وقایع و نظارت ۲۰
۲۰	۱۰.۷.۳	معماری و فرمت لاگ ها
	۷.۳ subsection	الگوریتمی چرخش فایل ۲۱
۲۱	۸.۳	تحلیل پیچیدگی الگوریتم ها
	۴ chapter	نتایج و ارزیابی ۲۴
۲۴	۱.۴	مقدمه
	۲.۴ section	محیط و ابزارهای آزمون ۲۴
۲۵	۳.۴	آزمون های عملکردی
	۳.۴ subsection	سناریوی ۱: حل نام بازگشتی ۲۵
۲۶	۲.۳.۴	سناریوی ۲: ناحیه محلی (Local Zone)
	۳.۴ subsection	سناریوی ۳: ارزیابی عملکرد کش ۲۶
۲۷	۴.۳.۴	سناریوی ۴: اعمال سیاست های امنیتی
	۴.۴ section	محدودیت ها و چالش های فنی ۲۷
۲۷	۱.۴.۴	بافر UDP
	۴.۴ subsection	مفسر جهانی پایتون ۲۸ (GIL)
۳۰	۵	جمع بندی و پیشنهادات
	۱.۵ section	مقدمه ۳۰
۳۰	۲.۵	جمع بندی دستاوردها
	۳.۵ section	بحث و چالش ها ۳۱
۳۱	۴.۵	پیشنهادهای برای بهبود و توسعه آتی
	۴.۵ subsection	تغییر ساختار داده سیاست ها ۳۱
۳۲	۲.۴.۵	پشتیبانی از پروتکل TCP
	۴.۵ subsection	انجام با پایگاه داده ۳۲
۳۲	۴.۴.۵	پروتکل های امنیتی مدرن
	chapter ۳۴	مراجع

فصل ۱

مقدمه

۱.۱ مقدمه

سامانه نام دامنه^۱ اغلب به عنوان «دفترچه تلفن اینترنت» توصیف می‌شود؛ زیرساختی حیاتی که نام‌های دامنه قابل خواندن توسط انسان (مانند google.com) را به آدرس‌های IP قابل پردازش توسط ماشین ترجمه می‌کند. در حالی که مفهوم DNS یکی از مباحث بنیادی در دروس شبکه‌های کامپیوتری است، پیاده‌سازی‌های معمول دانشگاهی اغلب به برنامه‌نویسی سوکت^۲ ساده محدود می‌شوند؛ یعنی ایجاد سروری که صرفاً به یک بسته گوش می‌دهد و پاسخی ثابت ارسال می‌کند.

با این حال، سرورهای DNS در دنیای واقعی در محیطی پرتراфик و گاه خصمانه فعالیت می‌کنند. آن‌ها باید قادر به مدیریت درخواست‌های همزمان^۳، مدیریت داده‌های محلی، ذخیره‌سازی پاسخ‌های بالادستی^۴ برای کاهش تأخیر، و دفاع در برابر سوءاستفاده‌ها باشند. هدف این پژوهش، پر کردن شکاف میان درک تئوریک از پروتکل DNS و یک پیاده‌سازی عملیاتی است.

¹Domain Name System (DNS)

²Socket Programming

³Concurrent Requests

⁴Caching

۲.۱ تعریف مسئله

در بستر آموزش مهندسی شبکه، دانشجویان غالباً سرورهای UDP تک‌رشته‌ای^۱ را پیاده‌سازی می‌کنند. اگرچه این پیاده‌سازی‌ها برای اثبات عملکرد پروتکل کافی هستند، اما از ضعف‌های معماری جدی رنج می‌برند:

- **مسدود شدن ورودی/خروجی^۲:** یک پرس‌وجوی گُند به سرور بالادستی^۳ می‌تواند کل سرور را مسدود کرده و باعث بروز خطا^۴ برای سایر کلاینت‌ها شود.

- **آسیب‌پذیری در برابر سیل درخواست‌ها^۵:** بدون شکل‌دهی ترافیک^۶ یا محدودسازی نرخ^۷، یک سرور UDP بدون وضعیت^۸ به سادگی با یک حمله انکار سرویس^۹ از کار می‌افتد و منابع سیستم (پردازنده و حافظه) را اشباع می‌کند.

- **فقدان نظارت و کنترل^{۱۰}:** اسکریپت‌های پایه معمولاً فاقد قابلیت فیلتر کردن محتوا یا اعمال سیاست‌های دسترسی هستند، که یکی از الزامات محیط‌های سازمانی یا کنترل والدین محسوب می‌شود.

این پژوهش با طراحی سروری که علاوه بر انطباق با پروتکل، از نظر عملیاتی مقاوم^{۱۱} باشد، به این مسائل می‌پردازد. هدف، جایگزینی راهکارهای صنعتی مانند BIND یا CoreDNS نیست، بلکه مهندسی سیستم آموزشی‌ای است که چالش‌های همزمانی، تأخیر و کنترل دسترسی را حل نماید.

۳.۱ اهداف تحقیق

هدف اصلی این گزارش پروژه، طراحی، پیاده‌سازی و ارزیابی یک سرور DNS سفارشی است. اهداف به دو دسته نیازمندی‌های عملکردی (آنچه سیستم انجام می‌دهد) و غیرعملکردی (چگونگی رفتار سیستم) تقسیم می‌شوند.

¹Single-threaded ²Blocking I/O ³Upstream ⁴Timeout ⁵Flooding ⁶Traffic Shaping
⁷Rate Limiting ⁸Stateless ⁹Denial of Service (DoS) ¹⁰Governance ¹¹Robust

۱.۳.۱ اهداف عملکردی

۱. حل‌کننده بازگشتی^۱: پیاده‌سازی یک حل‌کننده^۲ با قابلیت ارسال پرس‌وجوی دامنه‌های ناشناخته به یک تأمین‌کننده بالادستی (مانند 8.8.8.8) و بازگرداندن پاسخ به کلاینت.
۲. مدیریت ناحیه محلی^۳: توانمندسازی سرور برای عمل به عنوان سرور نام اصلی^۴ برای دامنه‌های داخلی (مانند myserver.local) با استفاده از فایل‌های پیکربندی ایستا.
۳. پشتیبانی از انواع رکوردها: پشتیبانی از رکوردهای حیاتی DNS شامل A (IPv4)، AAAA (IPv6)، CNAME (نام مستعار)، PTR (جستجوی معکوس) و TXT (متن).
۴. کش هوشمند^۵: پیاده‌سازی یک حافظه نهان درون برنامه‌ای^۶ برای ذخیره پاسخ‌ها با رعایت مقادیر طول عمر^۷ جهت کاهش ترافیک شبکه.

۲.۳.۱ اهداف غیرعملکردی

۱. همزمانی^۸: بهره‌گیری از معماری چندنخی (مدل نخ-به-ازای-درخواست^۹) برای اطمینان از اینکه پردازش‌های طولانی، حلقه اصلی اجرا را مسدود نمی‌کنند.
۲. امنیت و کنترل^{۱۰}: پیاده‌سازی دفاع‌های لایه کاربرد، مشخصاً یک محدودکننده نرخ^{۱۱} با الگوریتم پنجره لغزان^{۱۲} برای مقابله با حملات سیل‌آسا و یک موتور سیاست‌گذاری^{۱۳} برای لیست سیاه دامنه‌ها.
۳. تحمل خرابی^{۱۴}: تضمین پایداری سرور در برابر ورودی‌های ناقص یا قطعی شبکه بالادستی از طریق مدیریت خطای سراسری.

¹Recursive Resolution ²Resolver ³Authoritative Zone Management ⁴Primary Nameserver
⁵Smart Caching ⁶In-memory Cache ⁷TTL ⁸Concurrency ⁹Thread-per-Request ¹⁰Security and Governance
¹¹Rate Limiter ¹²Sliding Window ¹³Policy Engine ¹⁴Fault Tolerance

۴.۱ ساختار گزارش

ادامه این نوشتار به شرح زیر سازماندهی شده است:

فصل دوم (مبانی نظری و پیشینه تحقیق) به بررسی ساختار پروتکل DNS، مکانیسم‌های انتقال UDP و تئوری‌های الگوریتمی پشت شکل‌دهی ترافیک و کش‌سازی می‌پردازد.

فصل سوم (روش تحقیق / طراحی و پیاده‌سازی) جزئیات معماری ماژولار پروژه را تشریح می‌کند. در این فصل، پیاده‌سازی سرور UDP در پایتون، خط لوله حل نام^۱، مکانیسم کش ایمن در برابر نخ^۲ و ماژول‌های امنیتی واکاوی می‌شوند.

فصل چهارم (نتایج و ارزیابی) روش‌شناسی آزمون را ارائه می‌دهد. عملکرد سیستم از طریق ابزارهای استاندارد مانند dig و nslookup اعتبارسنجی شده و کارایی محدودکننده نرخ تحت بار شبیه‌سازی شده تحلیل می‌گردد.

فصل پنجم (جمع‌بندی و پیشنهادات) دستاوردهای پروژه را خلاصه کرده، محدودیت‌های ناشی از قفل مفسر جهانی پایتون^۳ را بحث می‌کند و بهبودهای آتی نظیر پشتیبانی از TCP و ادغام با پایگاه داده را پیشنهاد می‌دهد.

¹Resolution Pipeline

²Thread-safe

³Global Interpreter Lock (GIL)

فصل ۲

مبانی نظری و پیشینه تحقیق

۱.۲ مقدمه

پیش از تشریح پیاده‌سازی فنی سرور، ضروری است که مبانی نظری و استانداردهایی که این سیستم بر پایه آنها بنا شده است، تبیین گردد. این فصل به بررسی مکانیسم‌های سامانه نام دامنه^۱، پروتکل‌های انتقال لایه حمل^۲، و الگوریتم‌های بنیادین مورد استفاده برای مدیریت حافظه نهان^۳ و کنترل ترافیک می‌پردازد.

۲.۲ سامانه نام دامنه (DNS)

سامانه نام دامنه یک سیستم نام‌گذاری سلسله‌مراتب و غیرمتمرکز برای رایانه‌ها و سرویس‌های متصل به اینترنت است. این سامانه وظیفه اتصال اطلاعات مختلف (نظیر آدرس IP) به نام‌های دامنه را بر عهده دارد.^[۱]

^۱Domain Name System (DNS) ^۲Transport Layer ^۳Caching

۱.۲.۲ سلسله مراتب نام‌ها

فضای نام دامنه ساختاری درختی دارد. هر گره در این درخت متناظر با یک مجموعه منبع است. این سامانه به «ناحیه‌ها»^۱ تقسیم می‌شود که توسط سرورهای نام معتبر^۲ مدیریت می‌شوند:

- ناحیه ریشه^۳: بالاترین سطح درخت که توسط IANA مدیریت می‌شود.
- دامنه‌های سطح بالا^۴: نظیر .com، .org یا .ir.
- دامنه‌های سطح دوم: نظیر google.com یا university.ac.ir که معمولاً متعلق به سازمان‌ها هستند.

۲.۲.۲ ساختار بسته DNS

مطابق با استاندارد RFC 1035، پیام‌های DNS عمدتاً در بستر پروتکل UDP منتقل می‌شوند. یک پیام استاندارد شامل پنج بخش اصلی است:

۱. سرآیند^۵: شامل شناسه تراکنش^۶، پرچم‌ها (نظیر پرس و جو/پاسخ) و تعداد رکوردها در سایر بخش‌ها.
۲. پرسش^۷: نام دامنه و نوع رکورد درخواستی (مثلاً A یا AAAA).
۳. پاسخ^۸: رکوردهای منبع^۹ که به پرسش پاسخ می‌دهند.
۴. مرجع^{۱۰}: رکوردهایی که به سرورهای معتبر اشاره دارند.
۵. اطلاعات اضافی^{۱۱}: اطلاعات تکمیلی (مانند آدرس IP سرور مرجع) برای جلوگیری از پرس و جوهای اضافه.

¹Zones ²Authoritative Name Servers ³Root Zone ⁴Top-Level Domains (TLDs) ⁵Header
⁶Transaction ID ⁷Question ⁸Answer ⁹Resource Records (RRs) ¹⁰Authority ¹¹Additional

۳.۲ ملاحظات پروتکل انتقال

۱.۳.۲ پروتکل داده‌نگار کاربر (UDP)

سرویس DNS به طور سنتی از پروتکل UDP روی پورت ۵۳ استفاده می‌کند. UDP یک پروتکل «بدون اتصال»^۱ است، به این معنی که پیش از ارسال داده‌ها هیچ‌گونه دست‌تکانی^۲ صورت نمی‌گیرد.

• مزایا: تأخیر^۳ بسیار پایین و سربار کم که برای سرعت تفکیک نام دامنه حیاتی است.

• معایب: عدم تضمین تحویل بسته و فقدان مکانیزم کنترل جریان^۴، که سرور را در برابر حملات جعل آدرس آسیب‌پذیر می‌کند.

۲.۳.۲ محدودیت اندازه بسته

در استاندارد اولیه، پیام‌های UDP به ۵۱۲ بایت محدود بودند. پیام‌های بزرگتر نیازمند استفاده از پروتکل TCP بودند که سربار زمانی بالایی داشت. امروزه مکانیزم‌های توسعه‌یافته^۵ اجازه می‌دهند بسته‌های UDP تا ۴۰۹۶ بایت یا بیشتر حجم داشته باشند [۲]. سرور پیاده‌سازی شده در این پروژه از بافر ۴۰۹۶ بایتی برای سازگاری با این استاندارد مدرن استفاده می‌کند.

۴.۲ تهدیدات امنیتی

سرورهای DNS به دلیل نقش کلیدی و ماهیت بدون وضعیت UDP، اهداف جذابی برای حملات سایبری

هستند.

^۱Connectionless ^۲Handshake ^۳Latency ^۴Flow Control ^۵Extension Mechanisms for DNS (EDNS0)

۱.۴.۲ حملات انکار سرویس (DoS)

در یک حمله DoS استاندارد، مهاجم سیلی از درخواست‌ها را به سمت سرور روانه می‌کند. از آنجا که سرور برای پردازش هر بسته منابعی (مانند پردازنده و حافظه) را تخصیص می‌دهد، حجم بالای درخواست‌ها می‌تواند منجر به اشباع منابع و ناتوانی سرور در پاسخگویی به کاربران مجاز شود.

۲.۴.۲ حملات تقویتی^۱

از آنجا که پاسخ DNS معمولاً حجیم‌تر از پرسش آن است، مهاجمان می‌توانند با جعل آدرس IP قربانی، پرسش‌های کوچکی به سرور ارسال کنند تا سرور پاسخ‌های بزرگی را به قربانی بفرستد. اگرچه تمرکز این پروژه بر یک سرور محلی است، اما پیاده‌سازی «محدودیت نرخ»^۲ برای جلوگیری از سوءاستفاده‌های داخلی یک اصل مهندسی ضروری است.

۵.۲ استراتژی‌های ذخیره‌سازی نهان (Caching)

کش‌سازی مکانیسمی است برای ذخیره موقت پاسخ‌ها جهت پاسخگویی سریع‌تر به درخواست‌های آتی.

۱.۵.۲ زمان حیات^۳

هر رکورد DNS دارای یک مقدار «زمان حیات» (به ثانیه) است که توسط سرور مرجع تعیین می‌شود. یک حل‌کننده بازگشتی^۴ نباید رکورد را بیش از این زمان در حافظه نگه دارد تا از اعتبار داده‌ها اطمینان حاصل شود.

¹Amplification Attacks²Rate Limiting³Time-To-Live (TTL)⁴Recursive Resolver

۲.۵.۲ سیاست‌های تخلیه^۱

هنگامی که رکوردها منقضی می‌شوند، باید از حافظه حذف گردند:

- **تخلیه تنبل^۲:** رکورد تنها زمانی بررسی می‌شود که دوباره درخواست شود. این روش ساده است اما می‌تواند منجر به هدررفت حافظه شود.
- **تخلیه فعال^۳:** یک پردازش پس‌زمینه (زباله‌روب یا Janitor) به صورت دوره‌ای حافظه را پیمایش کرده و رکوردهای منقضی را حذف می‌کند. این روش در این پروژه برای تضمین مدیریت بهینه حافظه انتخاب شده است.

۶.۲ مبانی الگوریتمی

دو مفهوم الگوریتمی کلیدی، زیربنای کارایی و امنیت این پروژه را تشکیل می‌دهند.

۱.۶.۲ جداول درهم‌سازی^۴

برای پیاده‌سازی کش، دسترسی با مرتبه زمانی $O(1)$ ضروری است تا بررسی کش باعث کندی سیستم نشود. جداول درهم‌سازی (که در پایتون با dict پیاده‌سازی می‌شوند) زمان دسترسی، درج و حذف تقریباً ثابت را فراهم می‌کنند که برای نگاشت نام دامنه به رکوردهای منبع ایده‌آل است.

¹Eviction Policies²Lazy Eviction³Active Eviction⁴Hash Tables

۲.۶.۲ الگوریتم پنجره لغزان^۱

برای پیاده‌سازی محدودیت نرخ، از الگوریتم پنجره لغزان استفاده شده است. برخلاف «پنجره ثابت» که شمارشگر را در فواصل زمانی مشخص (مثلاً سر هر دقیقه) صفر می‌کند، پنجره لغزان نرخ درخواست‌ها را در یک بازه متحرک (مثلاً «یک ثانیه گذشته») ارزیابی می‌کند. به بیان ریاضی، اگر T زمان فعلی و W اندازه پنجره باشد، سرور تعداد درخواست‌های R را می‌شمارد به طوری که:

$$t_{request} \in [T - W, T]$$

اگر تعداد درخواست‌ها از حد مجاز بیشتر شود، بسته دور ریخته می‌شود. این روش توزیع ترافیک هموارتر و دقیق‌تری نسبت به شمارشگرهای ثابت ارائه می‌دهد.

¹Sliding Window

فصل ۳

طراحی و پیاده‌سازی

۱.۳ مقدمه

در این فصل، معماری نرم‌افزاری و جزئیات پیاده‌سازی سرور DNS با تمرکز بر چرایی و چگونگی تصمیمات طراحی تشریح می‌شود. هدف اصلی در طراحی این سیستم، ایجاد تعادل میان سادگی ساختار (برای اهداف آموزشی) و کارایی عملیاتی (برای محیط‌های واقعی) بوده است. بدین منظور از یک معماری ماژولار^۱ استفاده شده است که در آن هر مؤلفه مسئولیت مشخصی دارد.

۲.۳ معماری سیستم

سیستم از یک حلقه اصلی رویداد مبتنی بر سوکت^۲ تشکیل شده است که درخواست‌های ورودی را دریافت کرده و آن‌ها را به یک «حل‌کننده» مرکزی ارجاع می‌دهد.

^۱Modular Architecture ^۲Socket-based Event Loop

جریان داده‌ها به صورت زیر است:

۱. دریافت بسته UDP خام.

۲. بررسی توسط «محدودکننده نرخ» برای جلوگیری از حملات.

۳. پردازش توسط «حل‌کننده» (بررسی سیاست‌ها، ناحیه محلی، کش، یا ارسال به بالادست).

۴. ارسال پاسخ نهایی به کلاینت.

۱.۲.۳ ساختار پروژه

ساختار فایل‌های پروژه به گونه‌ای سازماندهی شده است که پیکربندی از منطق برنامه جدا باشد:

/dns-server		1
/config		2
server.conf.json	# Server settings (IP, Port)	3
policies.json	# Blocklist rules	4
zones.json	# Local authoritative records	5
/src		6
server.py	# UDP Network handling	7
resolver.py	# Core resolution logic	8
cache.py	# Thread-safe caching	9
interceptor.py	# Policy enforcement	10
ratelimit.py	# Traffic shaping	11
main.py	# Entry point	12

برنامه: 3.1

۳.۳ لایه شبکه: سرور چندنخی

پروتکل DNS عمدتاً بر بستر UDP عمل می‌کند. در یک پیاده‌سازی ساده تک‌نخی^۱، سرور باید یک درخواست را دریافت کرده، آن را پردازش کند و سپس به سراغ درخواست بعدی برود. اگر پردازش یک درخواست (مثلاً پرس‌وجو از یک سرور خارجی کند) طول بکشد، تمام درخواست‌های دیگر در صف انتظار مسدود می‌شوند.

۱.۳.۳ راهکار: مدل نخ-به-ازای-درخواست

برای حل این مشکل، کلاس UDPServer در فایل src/server.py از مدل همزمانی «نخ-به-ازای-درخواست»^۲ بهره می‌برد. در این مدل، حلقه اصلی سرور (_listen_loop) تنها وظیفه دریافت بسته خام را دارد. بلافاصله پس از دریافت، یک نخ (Thread) جدید ایجاد می‌شود که وظیفه پردازش آن بسته خاص را بر عهده می‌گیرد. این طراحی تضمین می‌کند که هیچ درخواست واحدی نمی‌تواند کل سرویس را متوقف کند.

```
def _listen_loop(self): 1
    while self.running: 2
        try: 3
            data, addr = self.sock.recvfrom(4096) 4
            client_thread = threading.Thread( 5
                target=self.handle_packet, 6
                args=(data, addr) 7
            ) 8
            client_thread.daemon = True 9
            client_thread.start() 10
        except Exception as e: 11
            self.logger.error(f"[-] Receive Error: {e}") 12
```

برنامه: 3.2

¹Single-threaded ²Thread-per-Request

۲.۳.۳ تحمل خرابی (Fault Tolerance)

یکی از جنبه‌های حیاتی سروورهای شبکه، پایداری در برابر ورودی‌های پیش‌بینی نشده است. در متد `handle_packet`، تمام عملیات پردازش درون یک بلوک مدیریت خطا (`try-except`) قرار گرفته است. این طراحی تضمین می‌کند که اگر یک بسته مخرب یا ناقص باعث بروز خطا در حین تجزیه (`Parsing`) شود، تنها همان نخ خاتمه می‌یابد و سرور اصلی بدون وقفه به کار خود ادامه می‌دهد. بدون این مکانیزم، یک خطای کوچک در پردازش یک بسته می‌توانست کل برنامه را متوقف کند.

۴.۳ هسته پردازشی: خط لوله حل نام

قلب تپنده سیستم، کلاس `DNSResolver` در فایل `src/resolver.py` است. این کلاس نقش «ارکستراسیون» را بازی می‌کند و تصمیم می‌گیرد که هر پرس‌وجو چگونه پاسخ داده شود. برخلاف اسکریپت‌های ساده که تنها یک کار انجام می‌دهند، این حل‌کننده یک خط لوله تصمیم‌گیری هوشمند را پیاده‌سازی می‌کند. در متد `process_query`، منطق برنامه به ترتیب زیر اجرا می‌شود:

۱. تجزیه بسته: ابتدا بایت‌های خام ورودی توسط کتابخانه `dnslib` به یک شیء ساخت‌یافته تبدیل می‌شوند تا نام دامنه (`qname`) و نوع رکورد (`qtype`) استخراج شود.

۲. لایه امنیتی: پیش از هر کاری، درخواست به `Interceptor` سپرده می‌شود. اگر دامنه در لیست سیاه باشد، یک پاسخ `REFUSED` تولید شده و پردازش متوقف می‌شود.

۳. لایه داده محلی: سپس فایل‌های محلی (`Zones`) بررسی می‌شوند. این اولویت‌بندی باعث می‌شود که مدیر سیستم بتواند حتی برای دامنه‌های عمومی (مثل `google.com`) رکوردهای محلی و سفارشی تعریف کند.

۴. لایه کش: اگر پاسخ در حافظه نهان موجود باشد، بلافاصله بازگردانده می‌شود تا از ترافیک شبکه کاسته شود.

۵. ارسال به بالادست: تنها در صورتی که هیچ‌یک از مراحل بالا نتیجه نداد، حل‌کننده یک سوکت جدید ایجاد کرده و درخواست را به سرور اینترنتی (مانند 8.8.8.8) ارسال می‌کند.

۱.۴.۳ ارسال ایزوله (Socket Isolation)

در هنگام ارسال درخواست به سرور بالادستی، برای هر تراکنش یک سوکت UDP جدید ایجاد می‌شود. این تصمیم طراحی باعث می‌شود که تداخلی بین نخ‌های مختلف رخ ندهد و مدیریت خطاهایی مانند Timeout برای هر درخواست به صورت مستقل انجام شود.

۵.۳ مدیریت داده‌ها: ناحیه محلی و کش

۱.۵.۳ پشتیبانی از رکوردهای متنوع

ماژول ZoneManager فایل zones.json را پارس کرده و در یک دیکشنری بارگذاری می‌کند. این ماژول فراتر از رکوردهای ساده A عمل کرده و از انواع زیر پشتیبانی می‌کند:

- **TXT**: برای توضیحات متنی یا احراز هویت سرویس‌ها.
- **CNAME**: برای ارجاع یک دامنه به دامنه دیگر.
- **PTR**: برای پاسخ‌دهی به پرس‌وجوهای معکوس (تبدیل IP به نام).

۲.۵.۳ کش هوشمند و ایمن

کلاس DNSCache در فایل src/cache.py پیاده‌سازی شده است. دو ویژگی کلیدی در طراحی این ماژول وجود دارد:

۱. ایمنی در برابر شرایط مسابقه^۱: از آنجا که صدها نخ ممکن است همزمان بخواهند در کش بنویسند یا از آن بخوانند، از یک قفل threading.Lock استفاده شده است. هرگونه دسترسی به دیکشنری کش تنها در داخل یک بلوک with self.lock انجام می‌شود.

```

# src/cache.py
1
def get(self, qname, qtype):
2
    key = (qname, qtype)
3
    with self.lock: # Thread-safety mechanism
4
        if key in self.cache:
5
            entry = self.cache[key]
6
            if time.time() < entry['expiry']:
7
                return entry['record']
8
            else:
9
                del self.cache[key] # Lazy eviction on access
10
    return None
11

```

برنامه ۳.۳:

۲. یادگیری معکوس خودکار^۲: این ویژگی یک بهینه‌سازی الگوریتمی است. هنگامی که سرور پاسخی برای google.com با آی‌پی 1.2.3.4 دریافت می‌کند، به طور هوشمندانه یک رکورد معکوس (PTR) نیز برای 4.3.2.1.in-addr.arpa تولید و ذخیره می‌کند. این کار باعث می‌شود اگر کلاینت بلافاصله درخواست Reverse DNS کند، پاسخ بدون نیاز به پرس‌وجوی مجدد آماده باشد.

^۱Race Condition ^۲Auto-Reverse Learning

۶.۳ امنیت و کنترل ترافیک (پیاده‌سازی دقیق)

این بخش حیاتی‌ترین قسمت دفاعی سیستم است که از اشباع منابع جلوگیری می‌کند.

۱.۶.۳ محدودسازی نرخ (Rate Limiting)

بدون کنترل نرخ، یک مهاجم می‌تواند با ارسال هزاران بسته در ثانیه، باعث ایجاد هزاران نخ در سرور شده و حافظه رم سرور را پر کند (Resource Exhaustion).

الگوریتم: پنجره لغزان^۱: ما از الگوریتم پنجره لغزان استفاده کرده‌ایم که دقیق‌تر از پنجره‌های ثابت است. منطق کار به شرح زیر است:

۱. برای هر IP، لیستی از زمان‌های درخواست (Timestamps) نگهداری می‌شود.
۲. با ورود هر درخواست جدید، ابتدا تمام زمان‌هایی که خارج از پنجره زمانی (مثلاً قدیمی‌تر از ۱ ثانیه) هستند، حذف (Prune) می‌شوند.
۳. تعداد زمان‌های باقی‌مانده شمارش می‌شود.
۴. اگر تعداد کمتر از حد مجاز (مثلاً ۱۰ عدد) باشد، درخواست پذیرفته و زمان فعلی به لیست اضافه می‌شود. در غیر این صورت، درخواست رد می‌شود.

¹Sliding Window

```
def is_allowed(self, ip): 1
    current_time = time.time() 2
    with self.lock: 3
        if ip not in self.clients: 4
            self.clients[ip] = [] 5
        6
        # 1. Prune timestamps outside the window 7
        timestamps = self.clients[ip] 8
        valid_timestamps = [t for t in timestamps 9
                            if current_time - t < self.window] 10
        11
        # 2. Check if under limit 12
        if len(valid_timestamps) < self.limit: 13
            valid_timestamps.append(current_time) 14
            self.clients[ip] = valid_timestamps 15
            return True # Allowed 16
        else: 17
            return False # Dropped 18
```

ratelimit.py

برنامه: 3.4

این ماژول در ابتدای چرخه پردازش سرور قرار گرفته است تا بسته‌های مخرب پیش از تخصیص منابع سنگین حذف شوند.

۲.۶.۳ سیاست‌گذاری و تطبیق پسوندی

ماژول DNSInterceptor در فایل `src/interceptor.py` وظیفه اعمال قوانین کنترلی را بر عهده دارد. این ماژول در هنگام راه‌اندازی، فایل `policies.json` را خوانده و قوانین را در حافظه بارگذاری می‌کند.

الگوریتم تطبیق: به جای تطبیق دقیق رشته‌ای (Exact Match)، از تطبیق پسوندی (Suffix Matching) استفاده شده است. در پایتون این کار با متد `endswith()` انجام می‌شود. این روش به سیستم اجازه می‌دهد تا با تعریف یک قانون والد، تمام فرزندان آن را پوشش دهد.

مثال عملی: اگر در فایل تنظیمات، دامنه `tiktok.com` به عنوان BLOCKED تعریف شود، سیستم به طور خودکار تمام زیردامنه‌های آن نظیر `ads.tiktok.com` و `cdn.tiktok.com` را نیز شناسایی کرده و مسدود می‌کند. این رفتار برای فیلتر کردن سرویس‌های تبلیغاتی و شبکه‌های اجتماعی که از هزاران زیردامنه استفاده می‌کنند، ضروری است.

۷.۳ سیستم ثبت وقایع و نظارت

برای نظارت بر عملکرد سرور، عیب‌یابی مشکلات و ثبت رویدادهای امنیتی، یک ماژول ثبت وقایع (Logging) در فایل `src/logger.py` پیاده‌سازی شده است. این سیستم به گونه‌ای طراحی شده است که وقایع را به صورت همزمان هم در کنسول (برای مشاهده آنی توسط توسعه‌دهنده) و هم در فایل (برای بایگانی و بررسی‌های بعدی) ثبت نماید.

۱.۷.۳ معماری و فرمت لاگ‌ها

در تابع `setup_logger`، فرمت استاندارد زیر برای تمامی پیام‌ها تعریف شده است:

TIMESTAMP | LEVEL | MESSAGE

برای مثال:

2025-01-20 14:30:05 | INFO | Request from 192.168.1.5: google.com

این فرمت استاندارد امکان تجزیه و تحلیل ماشینی لاگ‌ها را در آینده فراهم می‌کند. همچنین با استفاده از کتابخانه استاندارد logging پایتون و ماژول os، دایرکتوری مربوط به لاگ‌ها (logs/) به صورت خودکار در صورت عدم وجود ساخته می‌شود.

۲.۷.۳ استراتژی چرخش فایل (Log Rotation)

در سرورهای عملیاتی که ممکن است ماه‌ها روشن بمانند، نوشتن لاگ در یک فایل واحد منجر به پر شدن دیسک و دشواری در باز کردن فایل‌های حجیم می‌شود. برای رفع این چالش، از هندلر RotatingFileHandler استفاده شده است.

این هندلر با پارامترهای زیر پیکربندی شده است:

- **maxBytes=5MB:** به محض اینکه حجم فایل server.log به ۵ مگابایت برسد، سیستم نوشتن در آن را متوقف می‌کند.

- **backupCount=3:** فایل فعلی تغییر نام داده شده (مثلاً به server.log.1) و یک فایل خالی جدید ایجاد می‌شود. سیستم همیشه ۳ نسخه آخر را نگه داشته و قدیمی‌ترین نسخه را حذف می‌کند. این مکانیزم خودکار، پایداری عملیاتی سرور را بدون نیاز به دخالت مدیر سیستم تضمین می‌کند.

۸.۳ تحلیل پیچیدگی الگوریتم‌ها

کارایی سیستم بر اساس انتخاب ساختمان داده‌های مناسب تضمین شده است:

- **کش (Cache):** استفاده از دیکشنری پایتون (جدول درهم‌سازی) باعث شده است که پیچیدگی زمانی جستجو و درج به طور متوسط $O(1)$ باشد.
- **محدودکننده نرخ:** عملیات Prune بر روی یک لیست کوچک (به اندازه حد مجاز، مثلاً ۱۰ آیتم) انجام می‌شود، بنابراین پیچیدگی آن $O(K)$ است که K اندازه پنجره است و بسیار سریع اجرا می‌شود.

- **سیاست‌گذاری:** بررسی قوانین نیازمند پیمایش لیست سیاست‌ها است که پیچیدگی $O(N \cdot M)$ دارد (N تعداد قوانین، M طول دامنه). برای تعداد قوانین کم (زیر ۱۰۰۰) این روش سریع است، اما برای مقیاس‌های بزرگتر می‌توان در آینده از ساختار داده Trie استفاده کرد.

فصل ۴

نتایج و ارزیابی

۱.۴ مقدمه

در این فصل، عملکرد سرور DNS پیاده‌سازی شده مورد ارزیابی قرار می‌گیرد. هدف از این آزمون‌ها، اطمینان از صحت عملکرد ماژول‌های مختلف شامل حل‌کننده بازگشتی^۱، مدیریت ناحیه محلی، کش‌سازی و مکانیزم‌های امنیتی است.

۲.۴ محیط و ابزارهای آزمون

این پروژه قابلیت استقرار بر روی سرورهای ابری (Linux VPS) را دارا می‌باشد و آزمایش‌های اولیه در محیط راه دور انجام شده است. با این حال، نتایج ثبت شده در این گزارش جهت بررسی دقیق‌تر، در محیط توسعه محلی (Localhost) بر روی سیستم عامل لینوکس (Ubuntu) اخذ شده‌اند.

¹Recursive Resolver

برای ارسال درخواست‌ها و بررسی پاسخ‌ها از ابزارهای زیر استفاده شده است:

- **nslookup**: ابزار استاندارد خط فرمان برای پرس‌وجوی سرورهای نام و بررسی صحت رکوردهای بازگردانده شده.

- **اسکرپت پایتون**: جهت شبیه‌سازی ترافیک بالا و آزمون حمله DoS برای سنجش عملکرد محدودکننده نرخ.

سرور بر روی آدرس محلی 127.0.0.1 و پورت 53 پیکربندی و اجرا شده است.

۳.۴ آزمون‌های عملکردی

۱.۳.۴ سناریوی ۱: حل نام بازگشتی

در این آزمون، دامنه‌ای که در فایل‌های محلی سرور وجود ندارد (مانند google.com) درخواست می‌شود. انتظار می‌رود سرور درخواست را به بالادست (مثلاً 8.8.8.8) ارسال کرده و پاسخ را به کلاینت بازگرداند.

\$ nslookup google.com 127.0.0.1	1
	2
Server: 127.0.0.1	3
Address: 127.0.0.1#53	4
	5
Non-authoritative answer:	6
Name: google.com	7
Address: 142.250.180.206	8

nslookup

برنامه: 4.1

تحلیل: دریافت عبارت Non-authoritative answer و یک آدرس IP معتبر نشان می‌دهد که ماژول Resolver به درستی ارتباط با اینترنت را برقرار کرده و پاسخ را رله کرده است.

۲.۳.۴ سناریوی ۲: ناحیه محلی (Local Zone)

در این سناریو، رکوردی که در فایل zones.json تعریف شده است مورد پرسش قرار می‌گیرد. طبق پیکربندی، دامنه myserver.local باید به آدرس 127.0.0.1 اشاره کند.

\$ nslookup myserver.local 127.0.0.1	1
	2
Server: 127.0.0.1	3
Address: 127.0.0.1#53	4
	5
Name: myserver.local	6
Address: 127.0.0.1	7

برنامه 4.2:

این پاسخ ثابت می‌کند که سرور پیش از ارسال درخواست به اینترنت، دیتابیس محلی خود را بررسی کرده و اولویت‌بندی صحیح را رعایت می‌کند.

۳.۳.۴ سناریوی ۳: ارزیابی عملکرد کش

برای بررسی تأثیر کش، زمان پاسخگویی به درخواست‌های تکراری بررسی شد. اگرچه ابزار nslookup در لینوکس به طور پیش‌فرض زمان دقیق را نمایش نمی‌دهد، اما مشاهده رفتار سرور نشان می‌دهد:

- درخواست اول: با اندکی تأخیر (وابسته به سرعت اینترنت) پاسخ داده شد.

- درخواست دوم: پاسخ به صورت آنی (Instant) دریافت شد.

همچنین لاگ‌های سرور نشان می‌دهند که در بار دوم، نیازی به ایجاد سوکت جدید به سمت سرور بالادستی نبوده است که نشان‌دهنده عملکرد صحیح ماژول DNSCache است.

۴.۳.۴ سناریوی ۴: اعمال سیاست‌های امنیتی

در فایل policies_json، دامنه tiktok.com مسدود شده است. با ارسال درخواست برای این دامنه، انتظار می‌رود سرور پاسخ را رد کند.

\$ nslookup tiktok.com 127.0.0.1	1
	2
Server: 127.0.0.1	3
Address: 127.0.0.1#53	4
	5
** server can't find tiktok.com: REFUSED	6

برنامه: 4.3

پیام خطای REFUSED نشان می‌دهد که Interceptor با موفقیت درخواست را رهگیری کرده و بر اساس لیست سیاه، از پاسخ‌دهی امتناع ورزیده است.

۴.۴ محدودیت‌ها و چالش‌های فنی

با وجود عملکرد موفقیت‌آمیز، پیاده‌سازی فعلی دارای محدودیت‌هایی است که باید در نظر گرفته شوند:

۱.۴.۴ بافر UDP

سرور از یک بافر ثابت ۴۰۹۶ بیتی استفاده می‌کند. اگرچه این مقدار برای اکثر پاسخ‌های مدرن کافی است، اما طبق استاندارد، در صورتی که پاسخ بزرگتر از بافر باشد، سرور باید از پروتکل TCP استفاده کند که در نسخه فعلی پیاده‌سازی نشده است.

۲.۴.۴ قفل مفسر جهانی پایتون (GIL)

استفاده از نخ‌ها (Threads) در پایتون به دلیل وجود GIL، نمی‌تواند از تمام هسته‌های پردازنده به صورت موازی واقعی بهره‌برد. در ترافیک‌های بسیار سنگین (چندین هزار درخواست در ثانیه)، این موضوع می‌تواند گلوگاه پردازشی ایجاد کند.

فصل ۵

جمع‌بندی و پیشنهادات

۱.۵ مقدمه

در این فصل، خلاصه‌ای از فعالیت‌های انجام شده در طول این پروژه ارائه می‌گردد. همچنین با نگاهی نقادانه به سیستم پیاده‌سازی شده، محدودیت‌های موجود بررسی شده و پیشنهادهایی برای توسعه و بهبود کارایی سرور در پروژه‌های آتی مطرح خواهد شد.

۲.۵ جمع‌بندی دستاوردها

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک سرور DNS ماژولار و مقاوم با استفاده از زبان پایتون بود. در طول این مسیر، دستاوردهای زیر حاصل شد:

- **تحقق همزمانی:** با استفاده از مدل چندنخی، سروری طراحی شد که قادر است چندین درخواست را به صورت همزمان پردازش کرده و از مسدود شدن سیستم در اثر پاسخ‌های کند بالادستی جلوگیری کند.
- **مدیریت هوشمند داده‌ها:** پیاده‌سازی کش ایمن در برابر نخ با مکانیزم تخلیه فعال (Janitor Thread) و همچنین قابلیت یادگیری معکوس خودکار، منجر به بهینه‌سازی ترافیک و کاهش تأخیر گردید.

- امنیت و کنترل ترافیک: با استفاده از الگوریتم پنجره لغزان، لایه‌ای دفاعی برای مقابله با حملات انکار سرویس ایجاد شد که از اشباع منابع سرور جلوگیری می‌کند. همچنین قابلیت فیلترینگ دامنه‌ها امکان نظارت بر ترافیک شبکه را فراهم آورد.
- پایداری عملیاتی: پیاده‌سازی سیستم لاگینگ چرخشی و مدیریت خطای سراسری، پایداری سیستم را برای فعالیت‌های طولانی مدت تضمین کرد.

۳.۵ بحث و چالش‌ها

در طول پیاده‌سازی، چالش‌های فنی متعددی وجود داشت که در طراحی نهایی لحاظ شدند. یکی از مهم‌ترین چالش‌ها، مدیریت دسترسی همزمان به حافظه نهان بود که با استفاده از قفل‌های متقابل (Mutex Locks) مرتفع گردید. همچنین، به دلیل ماهیت بدون وضعیت (Stateless) پروتکل UDP، طراحی یک محدودکننده نرخ دقیق که بتواند بین کاربران عادی و حملات تمایز قائل شود، نیازمند دقت در انتخاب پارامترهای پنجره زمانی بود.

۴.۵ پیشنهادات برای بهبود و توسعه آتی

نسخه فعلی سرور اگرچه اهداف آموزشی و عملیاتی اولیه را پوشش می‌دهد، اما مسیرهای زیر برای ارتقای آن پیشنهاد می‌گردد:

۱.۴.۵ تغییر ساختار داده سیاست‌ها

در حال حاضر بررسی سیاست‌های مسدودسازی به صورت پیمایش خطی انجام می‌شود. برای پشتیبانی از لیست‌های سیاه بسیار بزرگ (مانند لیست‌های تبلیغات که شامل میلیون‌ها دامنه هستند)، پیشنهاد می‌شود از ساختار داده Trie یا فیلترهای بلوم^۱ استفاده شود تا زمان جستجو به $O(M)$ کاهش یابد.

^۱Bloom Filters

۲.۴.۵ پشتیبانی از پروتکل TCP

بسیاری از پاسخ‌های مدرن DNS به دلیل استفاده از امضاهای دیجیتال (DNSSEC) حجیم هستند. اضافه کردن لایه حمل TCP به سرور اجازه می‌دهد تا پاسخ‌های بزرگتر از ۴۰۹۶ بایت را نیز به درستی مدیریت کند.

۳.۴.۵ ادغام با پایگاه داده

برای مدیریت ناحیه‌های محلی در مقیاس وسیع، استفاده از فایل‌های JSON بهینه نیست. جایگزینی این فایل‌ها با پایگاه‌های داده‌ای نظیر SQLite یا Redis می‌تواند سرعت و قابلیت مدیریت داده‌ها را به شدت افزایش دهد.

۴.۴.۵ پروتکل‌های امنیتی مدرن

افزودن قابلیت‌های DNS over TLS (DoT) یا DNS over HTTPS (DoH) می‌تواند حریم خصوصی کاربران را در برابر شنود در شبکه محلی بهبود بخشد.

کتاب نامه

- [1] Mockapetris, P. Domain names - concepts and facilities, Nov 1987.
- [2] Damas, J. and Graff, M. Extension mechanisms for dns (edns(0)), Apr 2013.

